

Most jest strukturalnym wzorcem projektowym, którego celem jest oddzielenie abstrakcji od jej implementacji pozwalając obu warstwom na niezależną modyfikację. Wzorec ten znajduje zastosowanie w sytuacjach, gdy klasa posiada kilka niezależnych wymiarów (cech), np. kolor, kształt. Jeśli próbować to zrobić przez dziedziczenie, powstaje masa klas typu CzerwonyOkrag, NiebieskiKwadrat itd., co szybko robi chaos w kodzie i ciężko się w tym połapać.

Analogia:

Zamiast tworzyć sztywne klasy łączące obie cechy, dzielimy problem na dwie niezależne hierarchie: figury (abstrakcja) oraz Kolory (implementacja). Figura posiada "most" (referencje) do obiektu koloru. Dzięki temu możemy stworzyć nową figurę (np. Trojkat) bez definiowania jej kolorów, oraz nowy kolor (np. Zielony) bez modyfikowania klas figur. Struktura:

Abstrakcja – definiuje interfejs wysokopoziomowy i przechowuje referencje do obiektu typu implementacji. Rozszerzona abstrakcja – warianty bazowej abstrakcji, które rozbudowują jej logikę (np. wersja pilota z funkcją sterowania głosowego). Interfejs implementacji – wspólny interfejs dla wszystkich konkretnych implementacji; zazwyczaj deklaruje operacje elementarne. Konkretnie implementacje – zawierają specyficzny kod realizujący logikę platformowa lub techniczna. Klient – operuje wyłącznie na interfejsie abstrakcji, przekazując jej odpowiedni obiekt implementacji podczas inicjalizacji.

Implementacja:

należy wyodrębnić niezależne aspekty rozwoju klasy (np. typ figury). główna hierarchia klas (np. **Figura**) otrzymuje pole przechowujące referencje do interfejsu implementacji. metody klasy abstrakcji nie realizują logiki samodzielnie, lecz wywołują metody obiektu implementacji (delegowanie odpowiedzialności). klient łączy konkretną abstrakcję z wybraną implementacją.

Efekty stosowania

można dodawać nowe kształty bez ingerencji w logikę kolorów i odwrotnie. wprowadzanie nowych wariantów nie wymaga modyfikacji istniejącego kodu źródłowego.